connections open longer than you want and eating up database resources. If the `TeamFinder` fails, everything else that's part of the same flow in the application also fails. The controller throws an exception and can't continue. How do you safely separate the `TeamFinder` from the rest of the code?

It's time for another rewrite, and it doesn't look like the complexity is improving. You're now using four programming models: one for in-memory threads, one for database operations, one for the `Mentions` message queue, and one for the contacts web services.

How do you move from 3 servers to, say, 10, and then to 100 servers if need be? Obviously, this approach doesn't scale well: you have to change direction with every new challenge. In the next section, you'll find out if there's a design strategy that doesn't require you to constantly change direction.

1.5 Scaling with Akka

Let's see if it's possible to deliver on the promise to use only actors to meet the application's scaling requirements. Since it's probably still unclear to you what actors are, I'll use the terms *objects* and *actors* interchangeably and focus on the conceptual differences between this approach and the traditional one. Table 1.2 shows this difference in approaches.

Table 1.2 Actors compared to the traditional approach